Lecture 9: Introduction to C

CS32: Object Oriented Design and Implementation

Nabeel Nasir, PhD
UC Santa Barbara

April 30th 2026

Content based on slides from Ziad Matni

Second Half of the Course: Introduction to Systems Programming

- What is Systems Programming?
  - Programming close to the hardware and OS
  - Understanding how software interacts with:
    - Processor
    - Memory
    - Operating system

- Why Learn This?
  - Move from Big-O reasoning to **real hardware performance**
  - Understand how a program actually runs
    - What happens when you run ./a.out?
  - Build more **efficient, scalable, concurrent systems**
  - Maximize the utility of available hardware

Second Half of the Course: Introduction to Systems Programming

- Topics We Will Cover
  - C programming
  - Program execution & compilation pipeline
  - Computer Architecture and how a processor executes instructions
  - Operating Systems fundamentals
  - System calls & OS interfaces
  - OS Scheduling
  - Processes and threads
  - Concurrency and parallelism
- Why This Matters
  - Get you excited for other CS systems courses: 154, 170, 171, 176, 186
  - Industry demand shifting toward systems and performance engineering
  - Enables building high-performance infrastructure, not just applications

- **Key Objective**: understand how software, hardware, and the OS work together to execute and scale programs efficiently

Textbook

- Dive into Systems
  - By Suzanne J. Matthews, Tia Newhall, & Kevin C. Webb.
  - Available online: https://diveintosystems.org/book

Agenda

- C programming
- Compiling, linking
- I/O
- Structs
- Memory
- and more!

Intro to C programming

- What is C?
  - Procedural, low-level, manual memory
- Why learn C?
  - Systems programming
  - OS, embedded, compilers
- C vs C++
  - No classes
  - No reference variables
  - No STL
  - Manual memory always
- Closest to a standard reference document: https://cppreference.com/w/c.html

About C standards

- C89, C95, C99, C11, C17, C23
- We'll be using the **C99 standard**
- How to check C standard being used:

```
gcc -dM -E - < /dev/null | grep __STDC_VERSION__
```

  - 199409L → C95
  - 199901L → C99
  - 201112L → C11
  - 201710L → C17
  - 202311L → C23

If you want to follow along with code examples: **lec09-files.zip** on Canvas

Basic Compiling & Linking with gcc

- gcc translates your C source (.c) into an executable binary
- The 4 Main Steps:

```
gcc -o myprog myprog.c
./myprog
```

1. Preprocess:
   - Expand #include, #define, etc.
2. Compile:
   - Translate the C program source code to machine-specific assembly code
3. Assemble:
   - Produce relocatable binary object code (.o)
4. Link:
   - Combine your object code with libraries into an executable
   - e.g., the linker will find the `printf` function in the standard C library (`libc.so`)

Basics I/O in C — printf & scanf

- What these do
- printf — print formatted output to the screen
- scanf — read formatted input from the user
- (both come from <stdio.h>)

```
#include <stdio.h>
```

printf: print formatted data to stdout

```
int printf ( const char * format, ... );
```

- The format string often contains
  - special format specifiers,
  - tabs (\t) or newlines (\n)
  - placeholders for values in the output (% followed by a type specifier).

printf.c

```c
int x = 5, y = 10;
float pi = 3.14;

printf("x is %d and y is %d\n", x, y);

printf("%g \t %s \t %d\n", pi, "hello", y);
```

```
%f, %g: placeholders for a float or double value
%d:     placeholder for a decimal value (char, short, int)
%u:     placeholder for an unsigned decimal
%c:     placeholder for a single character
%s:     placeholder for a string value
%p:     placeholder to print an address value
```

scanf — Input

```
int scanf ( const char * format, ... );
```

- Similar placeholder in format string
- scanf needs the address of a variable
  - always use &

```
int x;
float pi;

// read in an int value followed by a float value ("%d%g")
// store the int value at the memory location of x (&x)
// store the float value at the memory location of pi (&pi)
scanf("%d%g", &x, &pi);
```

```
scanf("%d", x); // WRONG!
```

Introduction to Strings and the C String Library

- For strings: can't assume a string's length = size of array
  - Strings in C must end with a null character '\0' => null-terminated

- char arrays must be declared with enough capacity to store \0
  - Example: to store the string "hi", you need a `char str[3] = "hi";`
    - str[0] = 'h', str[1] = 'i', str[2] = '\0'

strlen and strcpy

```
#include <stdio.h>
#include <string.h>   // include the C string library

int main(void) {
    char str1[10];
    char str2[10];
    int len;

    str1[0] = 'h';
    str1[1] = 'i';
    str1[2] = '\0';

    len = strlen(str1);

    printf("%s %d\n", str1, len);  // prints: hi 2

    strcpy(str2, str1);     // copies the contents of str1 to str2
    printf("%s\n", str2);   // prints:  hi

    strcpy(str2, "hello");  // copy the string "hello" to str2
    len = strlen(str2);
    printf("%s has %d chars\n", str2, len);   // prints: hello has 5 chars
}
```

C Structs

- A struct groups related data together.
- Unlike C++, a struct in C:
  - Has **no methods**
  - Has **no constructors**
  - Is just **data**
- Fields are stored **in order**
- Stored **contiguously** in memory
- `sizeof(SomeStruct)` tells you total size

```
struct studentT {
    char  name[64];
    int   age;
    float gpa;
    int   grad_yr;
};
```

```
struct studentT student1, student2;

// access field values using .
strcpy(student1.name, "Ruth");
student1.age = 17;
student1.gpa = 3.5;
student1.grad_yr = 2021;
```

C Structs with pointer

- . → direct access
- -> → pointer access

```c
struct studentT {
    char  name[64];
    int   age;
    float gpa;
    int   grad_yr;
};
```

```c
// the grad_yr field of what sptr points to gets 2021:
(*sptr).grad_yr = 2021;

// the age field of what sptr points to gets s.age plus 1:
(*sptr).age = s.age + 1;
```

```c
// the gpa field of what sptr points to gets 3.5:
sptr->gpa = 3.5;

// the name field of what sptr points to is a char *
// (can use strcpy to init its value):
strcpy(sptr->name, "Lars");
```

Typedef to simplify C struct instantiation

- Without typedef, you must write: `struct Point p;`

- To simplify:

```
typedef struct {
    int x;
    int y;
} Point;
```

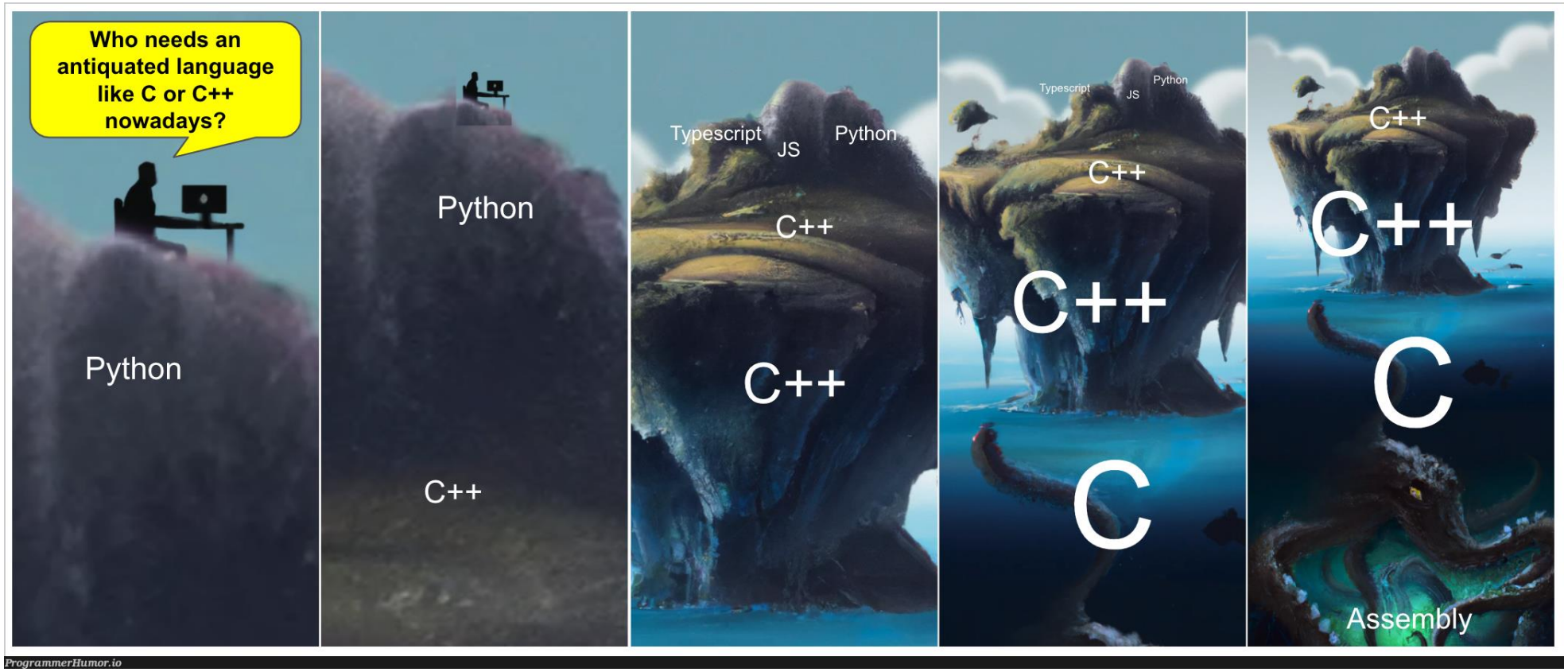
- Now you can write: `Point p;`

- typedef creates an alias for a type
  - It does NOT create a new type, just renames it

Break!

Questions?

Join at
slido.com
#2265 987



Dynamic Memory Allocation in C

- Handled using these functions: `malloc, calloc, realloc, free`
- `malloc`: allocate a single block of contiguous memory on the heap at runtime.
  - The memory allocated by `malloc` is uninitialized (can be garbage values)
  - Returns a void * to starting address of the allocated memory, if successful
  - Returns null pointer if failed

- Example: if we want to create an array to store 5 ints
  - Since the size of int is 4 bytes, we need 5 * 4 bytes = 20 bytes of memory

```
int *ptr = (int *)malloc(sizeof(int) * 5);

if (ptr == NULL) {
    printf("Allocation Failed");
    exit(0);
}
```

Dynamic Memory Allocation in C

- `calloc`: similar to `malloc`, but it initializes the allocated memory to **zero**

```
int *ptr = (int *)calloc(5, sizeof(int));
```

- `realloc`: used to resize a previously allocated memory block
  - Can resize it to a smaller or larger block

```
int *ptr = (int *)malloc(5 * sizeof(int));
// Resize the memory block to hold 10 integers
ptr = (int *)realloc(ptr, 10 * sizeof(int));
```

- `free`: used to release dynamically allocated memory back to the allocator

```
int *ptr = (int *)calloc(5, sizeof(int));
// Do some operations.....
free(ptr);
```

Dynamically Allocating Strings

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(void) {
    int size;
    char str[64];         // statically allocated
    char *new_str = NULL; // for dynamically allocated

    strcpy(str, "Hello");
    size = strlen(str);   // returns 5

    new_str = malloc(sizeof(char) * (size+1)); // need space for '\0'
    if(new_str == NULL) {
        printf("Error: malloc failed!  exiting.\n");
        exit(1);
    }
    strcpy(new_str, str);
    printf("%s %s\n", str, new_str);    // prints "Hello Hello"

    strcat(str, " There");  // concatenate " There" to the end of str
    printf("%s\n", str);    // prints "Hello There"

    free(new_str);  // free malloc'ed space when done
    new_str = NULL;

    return 0;
}
```

What is a void pointer (void*)?

- Pointers in C all take up the same space in memory, regardless of type
  - 4 bytes on 32 bit system, 8 bytes on 64 bit
- Possible to specify a generic pointer - can point to any type, or unspecified type

- void* cannot be dereferenced like a normal pointer
  - Compiler knows the starting addr of some value on memory
  - Compiler has no idea how much space the value takes up in memory!
  - Could be an int (4 bytes), or char (1 byte), or something else entirely
  - void* has to be explicitly re-casted to another pointer type to be dereferenced

Why is void* useful?

- Generic parameters and return values in functions
  - e.g. `malloc`
    - Allocates a specified amount of memory on heap
    - Makes no assumptions about what type you'll store on the allocated memory -> return `void*`
    - Have to explicitly recast the `void*`, so you can dereference and use the memory

```
int *array = (int*) malloc(sizeof(int) * 10);
```

Memory Manipulation: memset

```
void *memset(void *ptr, int x, size_t n);
```

- Used to set all BYTES equal to x in a contiguous block of memory
- Block of memory starts from addr in `ptr`, and continues for n bytes
- ptr is a `void*`, can pass any pointer into `memset`

- Set all elements of array equal to 0:

```
int arr[n];
memset(arr, 0, n*sizeof(arr[0]));
```

Side note about memset

- Set all elements of array equal to 8?

```
int arr[n];
memset(arr, 8, n*sizeof(arr[0]));
```

- Doesn't work! `memset` sets values by BYTE, and an int is 4 bytes
- Every int has their four bytes each set to the binary representation of 8, which turns into…
- 00001000 00001000 00001000 00001000 = 134744072!!!

Memory Manipulation: memcpy

```
void *memcpy(void *to, void *from, numBytes);
```

- Copies a specified number of bytes from one place in memory to another
  - Source starts at addr in **`to`**, destination starts at addr in **`from`**
- Simply copies raw data byte by byte, does not handle data at a higher level
  - Does not perform memory initialization
  - No regard for types or type conversions
  - Shallow copy of objects

File I/O

- <stdio.h> defines several file I/O functions
  - `fopen, fclose`
  - `fseek`
  - `fprintf, fscanf`
  - `fread, fwrite`

- Also defines file pointers FILE *

File Pointers (FILE *) and fopen

- Files are treated as input and output streams in C
- `FILE *` refers to a specific stream when passed into file I/O functions
- Set a file pointer using `fopen`

```
FILE *file = fopen(filepath, mode);
```

  - `filepath` is a string representing relative or absolute file path
  - `mode` is a string representing the purpose of the file
    - "r" - read mode
    - "w" - write mode
    - "a" - append mode

- `fopen` returns NULL if something goes wrong with opening the file

fseek

- File streams in C keep track of your position within a file
  - When you first open a file, your position will be at the first character
  - Reading or writing characters moves your position forward
    - If you want to get to the 10th character, you can read the first 9 characters, or… use fseek!

```
fseek(FILE *f, long offset, int whence);
```

- `f` = the file, as a file pointer
- `offset` = how many characters to move, relative to a given position
  - Positive to move forward, negative to move backwards
- `whence` = position to offset relative to
  - `SEEK_SET`: Beginning of the file
  - `SEEK_CUR`: Your current position
  - `SEEK_END`: End of the file

```
fseek(f, -3, SEEK_END); // seek 3 chars back from end of file
```

Reading and writing to files

- We have `scanf` and `printf` for r/w to console, and we have counterparts of these functions for r/w to files!

```
int fscanf(FILE *f, char *format, ...);
int fprintf(FILE *f, char *format, ...);
```

- Functions are very similar, you just need to specify a file

Reading and writing to files

- We also have functions to r/w to arrays

```
size_t fread(void *buffer, size_t size, size_t count, FILE *stream);
size_t fwrite(void *buffer, size_t size, size_t count, FILE *stream);
```

- `buffer` = array to read into/write from
- `size` = size of a SINGLE element in the array
  - Need to specify because buffer is a generic void*
- `count` = number of elements to read/write
  - Doesn't have to be the size of the entire array
- `stream` = file pointer of file to read/write

Reading and writing to files: examples

```
int buffer_a[5];
fread(buffer, sizeof(int), 5, file);
```

Read five ints from the file stream to `buffer_a`

```
char buffer_b[5];
fwrite(buffer_b, sizeof(char), 2, file);
```

- Write two chars from `buffer_b` to the file stream

fclose

- Be sure to close your file streams with fclose once you're done using them!

```
fclose(FILE *fp);
```

- Why close?
  1. "Buffer" Danger (Data Loss)
     - C doesn't write to hard drive every time you call fprintf or fwrite => incredibly slow
     - Instead stores data in a buffer
     - With fclose: buffer is "flushed" => all pending data is finally pushed to disk
     - Without fclose: if program ends abruptly or crashes, data still sitting in buffer might never be written
  2. Resource Leaks (The "Too Many Open Files" Error)
     - Every time you open a file, OS assigns a File Descriptor (an integer index) to your process.
     - Computers have a strict limit on how many files one program can have open at once
     - Once you hit that limit, every future fopen call will fail and return NULL.

cmd line args

- We are familiar with the regular main() function…
- But, we can also have a main function header that looks like this:

```
int main(int argc, char *argv[]){
```

  - This supports command line arguments!

- argc = number of arguments
- argv = an array of char*, each represents one cmd line arg as a string
- Last elem in argv is NULL to signal the end of the list of cmd line args

cmd line args

- Let's say you have an executable from C program `pgm`
- You can execute it with command line args like so:

```
./program hello 10 "quotes for spaces"
```

- What will be the values of `argc` and `argv`?
  - argc = 4
  - argv[0] = "./program"
    - Important: The actual command counts as a argument!
  - argv[1] = "hello"
  - argv[2] = "10"
    - Note that this is a C-string "10", NOT a numerical value!
    - Numerical values have to be converted
    - e.g. `int num = atoi(argv[2]);`
  - argv[3] = "quotes for spaces"
  - argv[4] = NULL

perror

- Defined in <stdio.h>
- Used to print out a custom message for an error code
- C stores error codes in a system variable errno when something goes wrong

```
int main() {
    FILE *fptr = fopen("example.txt", "r");
    if(fptr == NULL) {
      perror("fopen");  // If this fails, it prints: "fopen: [The actual reason]"
    }
    return 0;
}
```

- If fopen fails because the file doesn't exist, this will be printed:

```
An error occurred: No such file or directory
```

Thank you!